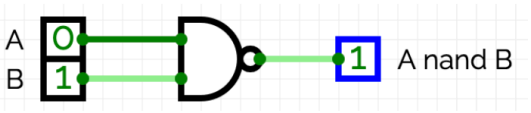
Learning Objectives

The primary objective of this worksheet is to gain practical experience with CircuitVerse while applying the concepts introduced in the lecture videos. Students construct and experiment with digital circuits rather than merely analyzing them on paper.

The following secondary objectives are also addressed:
- developing an understanding of the mathematical modeling of logic gates and recognizing the limitations of this model (in particular, propagation delays),
- experimentally introducing the concept of two's complement arithmetic,
- distinguishing between combinational and sequential logic through hands-on experiments.

Step 1

First, complete the truth table for a NAND gate.

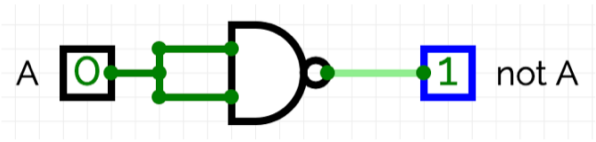


A	B	$A \barwedge B$
0	0	
0	1	
1	0	
1	1	

Background: A NAND gate (short for NOT-AND, symbol: $\barwedge$) is a universal building block: every other logic gate can be constructed using only NAND gates. This property is not only mathematically interesting but also practically important, since a single gate type is sufficient to implement a wide variety of digital circuits.

NOT Gate: First, verify that the truth table for the expression $A \barwedge A$ is correct and that the expression is equivalent to the logical negation of A. Then implement the circuit shown below in CircuitVerse.

A	$A \barwedge A$
0	1
1	0



Next, complete the truth tables for the following expressions. Afterwards, implement each circuit in CircuitVerse.

AND Gate Using Two NAND Gates

A	B	$A \barwedge B$	$(A \barwedge B) \barwedge (A \barwedge B)$
0	0		
0	1		
1	0		
1	1		

OR Gate Using Three NAND Gates

A	B	$A \barwedge A$	$B \barwedge B$	$(A \barwedge A) \barwedge (B \barwedge B)$
0	0			
0	1			
1	0			
1	1			

Step 2

An XOR gate (symbol: $\dot{\vee}$) can be expressed by the following equation:

$$A \mathbin{\dot{\vee}} B = \big(B \mathbin{\barwedge} (A \mathbin{\barwedge} A)\big) \mathbin{\barwedge} \big(A \mathbin{\barwedge} (B \mathbin{\barwedge} B)\big)$$

Verify this by completing the following truth table.

A	B	$A \mathbin{\barwedge} A$	$B \mathbin{\barwedge} (A \mathbin{\barwedge} A)$	$B \mathbin{\barwedge} B$	$A \mathbin{\barwedge} (B \mathbin{\barwedge} B)$	$\big(B \mathbin{\barwedge} (A \mathbin{\barwedge} A)\big) \mathbin{\barwedge} \big(A \mathbin{\barwedge} (B \mathbin{\barwedge} B)\big)$

Show, again by completing a truth table, that the following equivalent expression also implements XOR:

$$A \mathbin{\dot{\vee}} B = \big((A \mathbin{\barwedge} B) \mathbin{\barwedge} A\big) \mathbin{\barwedge} \big((A \mathbin{\barwedge} B) \mathbin{\barwedge} B\big)$$

A	B	$A \mathbin{\barwedge} B$	$(A \mathbin{\barwedge} B) \mathbin{\barwedge} A$	$(A \mathbin{\barwedge} B) \mathbin{\barwedge} B$	$\big((A \mathbin{\barwedge} B) \mathbin{\barwedge} A\big) \mathbin{\barwedge} \big((A \mathbin{\barwedge} B) \mathbin{\barwedge} B\big)$

Tasks

- Implement both XOR expressions in CircuitVerse.
- Explain to a partner the practical advantage of the second implementation.
- Additional challenge: Try to transform the first equation into the second one formally. You may use the identity $A \mathbin{\barwedge} B = 1 - A \cdot B$ to rewrite the logical expression as an algebraic equation.

Step 3

Note: Once you have completed this step, let us know. We have brought a laboratory power supply, several logic gates, and an oscilloscope, and would like to show you how the behavior you have been simulating actually looks in a real electronic circuit.

Background: Describing logic gates using Boolean functions is a mathematical model. In this model, a logic gate is assumed to produce its output instantaneously. In reality, however, the output changes only after a short delay, typically on the order of nanoseconds. A more accurate model can be obtained using differential equations that describe how analog quantities, such as voltages and currents, evolve in an electronic circuit. A digital value is then assigned to an analog quantity such as the voltage:

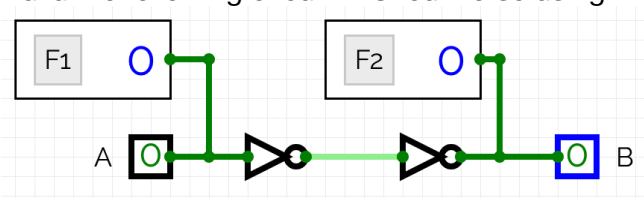
- If the voltage is below a threshold U_0, the signal is interpreted as `0`.
- If the voltage is above a threshold U_1, the signal is interpreted as `1`.
- Since the thresholds satisfy $U_0 < U_1$, these two cases are mutually exclusive. If neither applies, the digital state is undefined (usually denoted by `x`). After a certain amount of time—the propagation delay of the logic gate—the output will always settle to either `0` or `1`.

For this session, modeling logic gates with Boolean functions is entirely sufficient. Later in the course, however, we will distinguish between *combinational logic*, which is the focus of today's worksheet, and *sequential logic*, which we will use to build memory elements such as flip-flops. When modeling sequential logic, the propagation delays that we are currently ignoring become important.

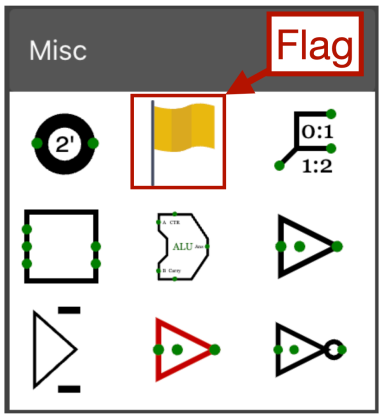
So, for now, consider this a small investment in your future neural network. 😉

Instructions

Build the following circuit in CircuitVerse using two NOT gates.

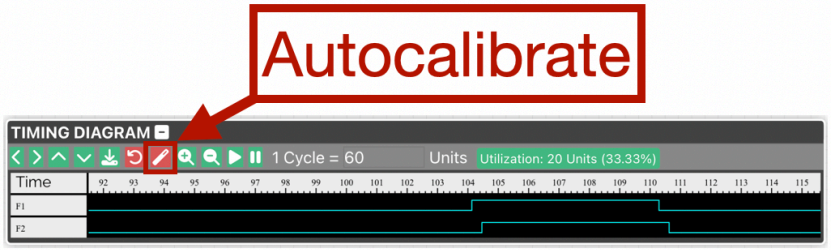


The components labeled **F1** and **F2** are Flags, which can be found under **Misc**. Their names can be changed in the **Properties** panel using the **Debug Flag Identifier setting**.



If we model the circuit using Boolean functions, the input `A` and the output `B` always have the same value. To observe that real logic gates exhibit a propagation delay, open the Timing Diagram in CircuitVerse:

- Click Autocalibrate in the Timing Diagram.



- Change the value of input `A` from `0` to `1`, and then back from `1` to `0`.

The Timing Diagram displays the values observed at F1 and F2 as functions of time. By default, every logic gate in CircuitVerse has a propagation delay of 10 ns. You can change this value in the Properties panel of each gate.

Step 4

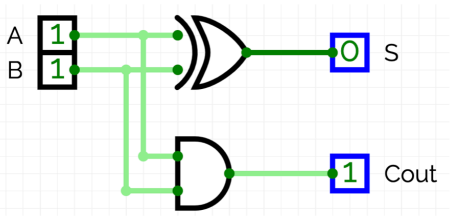
Next, we investigate how propagation delays affect the behavior of our 4-bit adder.

An interesting case occurs when the bit pattern `1111` is added to `0001`. The correct result is `0000` with a carry-out (`Cout`) of `1`. However, if we observe the output bits during the computation, the result first becomes `1110`, then `1100`, then `1000`, and only afterwards settles to the correct value `0000`.

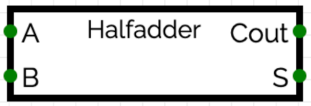
In practice, propagation delays limit the maximum clock frequency of a processor. Depending on the length of the longest logic paths, it may be necessary to wait longer or shorter before the result of one computation can safely be used as the input to the next.

To observe this effect, reconstruct the 4-bit adder presented in the lecture videos. Then use Flags and the Timing Diagram to visualize how the signals propagate through the circuit.

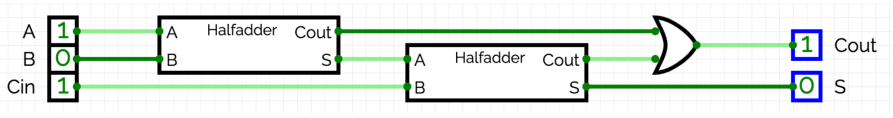
Half Adder



Layout:

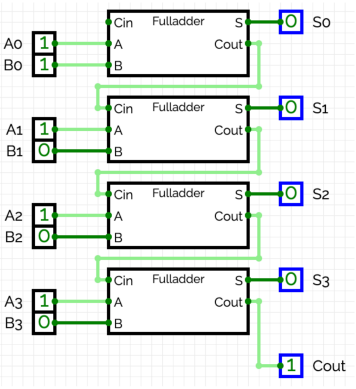


Full Adder

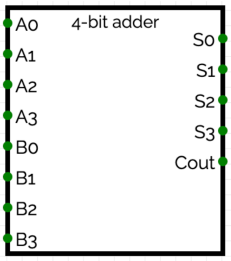


Layout:

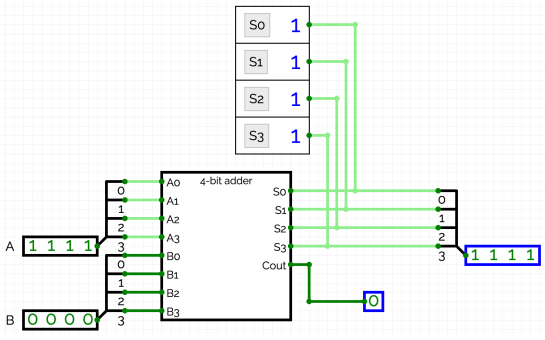
4-bit Adder

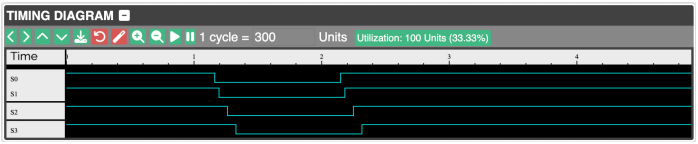


Layout:



Experiment





Our 4-bit adder is a *ripple-carry adder*. More sophisticated adder architectures exist—for example, the *carry-lookahead adder*. These designs compute the carry bits more efficiently, allowing the final result to become available sooner. Different adder architectures also differ in the number of logic gates required for their implementation. Understanding the trade-offs between these architectures is important when selecting the most appropriate design for a particular application.

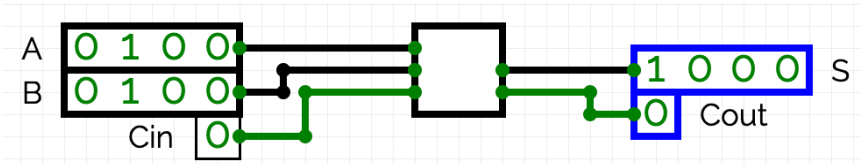
Step 5

For the remainder of this session, we will focus exclusively on the ripple-carry adder.

You may have noticed that our 4-bit adder consists of four full adders. In fact, however, the least significant stage does not require a carry input. It could therefore be implemented using a half adder followed by only three full adders. One advantage of the current design is that two identical 4-bit adders can easily be combined to form an 8-bit adder.

Task

CircuitVerse provides a ready-made **Adder** component under Misc. Configure it as a **4-bit adder** in the Properties panel.



Use two 4-bit adders to construct an 8-bit adder.

Schritt 6

A 4-bit adder does not compute $a + b$ directly. Instead, it computes

$$a \oplus b := (a + b) \bmod 16.$$

With respect to this operation, the set

$$G = \{0,1\}^4$$

forms a group. Consequently, every $a \in G$ has a unique additive inverse, denoted by $-a$, satisfying

$$a \oplus (-a) = 0$$

where 0 denotes the bit pattern consisting entirely of zeros.

The practical consequence is that subtraction can be reduced to addition. To subtract two bit patterns, it is therefore sufficient to determine the bit pattern representing $-a$. A simple rule can be derived:

- Adding the bit pattern `0001` to `1111` yields `0000`.
- If you add any bit pattern (for example `1010`) to its bitwise complement (in this case `0101`), the result is always `1111`. If you then set `Cin` to `1`, the result always becomes `0000`.

Task

- Implement a logic circuit for subtraction in CircuitVerse. Use a 4-bit NOT gate to invert the bit pattern of `b`. Just like addition, the circuit computes

  $$a \ominus b := (a - b) \bmod 16$$

  rather than $a - b$.
- Modify the circuit so that `Cout` is `0` if

  $$a - b = a \ominus b$$

  and 1 otherwise.

Remark: The representation introduced here is known as two's complement.

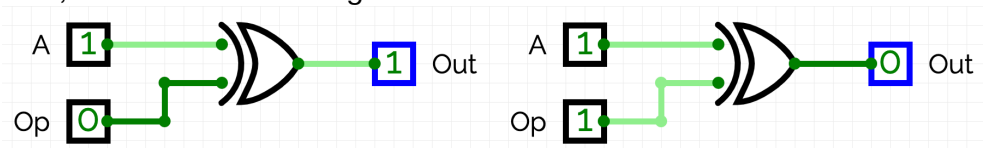
Other representations exist as well, for example **one's complement** and **sign-and-magnitude**. Feel free to read about them on your own or ask us about them during the session.

One reason why two's complement is almost universally used in modern processors for integer arithmetic is its elegant mathematical structure: the set of n-bit patterns forms a group under addition modulo 2^n. In contrast, representations such as one's complement and sign-and-magnitude require special handling because they contain two different representations of zero (`+0` and `-0`). As a result, the bit patterns no longer form a group under the corresponding addition.

Step 7

In this step, extend your circuit so that it can perform both addition and subtraction.

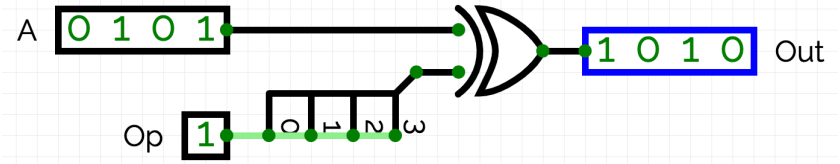
First, consider the following circuit:



The input Op determines whether the input `A` is inverted. A truth table shows that the circuit behaves as follows:

- If `Op = 0`, then `Out = A`.
- If `Op = 1`, then `Out = ¬A`.

The following circuit applies the same idea to a 4-bit input by using a 4-bit XOR gate:



Use this technique to selectively invert one operand, allowing the circuit to switch between addition and subtraction.

Notice that the same signal Op can be connected to both the XOR gate and the Cin input of the adder. Consequently,

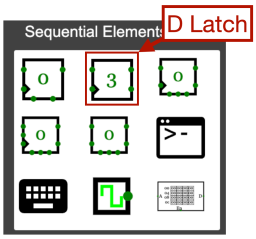
- `Op = 0` performs addition, and
- `Op = 1` performs subtraction.

Step 8

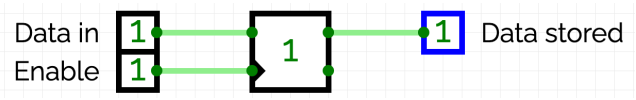
Note: The last three steps of this worksheet are intended as a gentle preview of sequential logic. We will return to this topic in later sessions and explain it in much greater detail. For now, the goal is simply to become familiar with some of the basic ideas—you will find them much easier to understand once you have seen them before.

Note: Once you have completed this step, let us know. We have brought breadboards containing D latches built entirely from NAND gates (each NAND gate itself being constructed from two NPN transistors). We'd be happy to show you how the circuit behaves in real hardware.

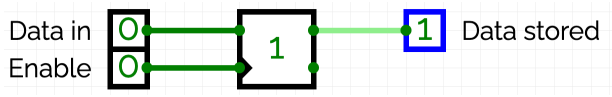
To build a computer, we need storage elements that can hold intermediate results so they can be used in subsequent computations. Later in the course we will use D flip-flops for this purpose. As a first step, however, we consider the simpler D latch, which is available in CircuitVerse under Sequential Elements.



We can derive the behavior experimentally using the following circuit:



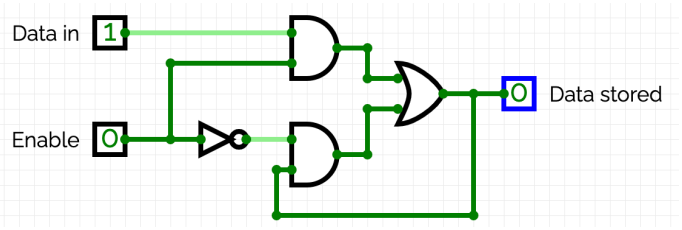
When experimenting with the circuit, you should notice the following behavior:
- If **Enable** is `1`, **Data stored follows Data in**.
- If **Enable** is `0`, **Data stored retains its previous value**, regardless of the value of Data in.



The behavior of the D latch can be characterized by the implicit equation

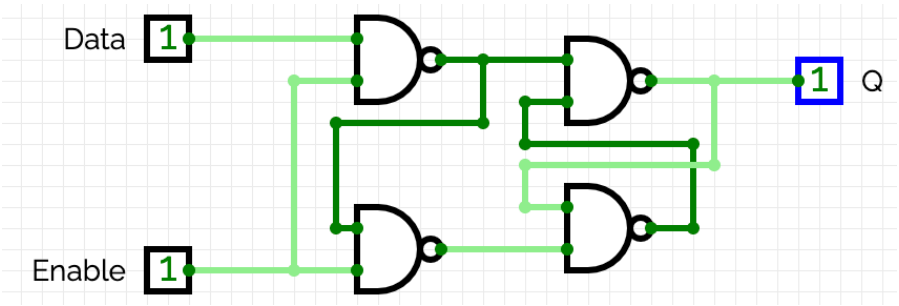
$$\text{Data}_{\text{stored}} = \text{Data}_{\text{stored}} \wedge \neg\text{Enable} \vee \text{Data}_{\text{in}} \wedge \text{Enable}$$

Since this equation is implicit, it cannot be solved explicitly for $\text{Data}_{\text{stored}}$. Consequently, a D latch cannot be described by a Boolean function alone. Nevertheless, we can try to implement the equation directly using logic gates:



Tasks

- Implement the circuit in CircuitVerse. Verify experimentally that it behaves like the built-in D latch.
- Change the propagation delay of the NOT gate to 20 ns. The circuit should no longer behave like a D latch.
- The equation above characterizes the behavior of a D latch, but it does not tell us how to implement one. A stable implementation must take propagation delays into account. We will not study this in detail in this course, but one possible implementation is shown below.

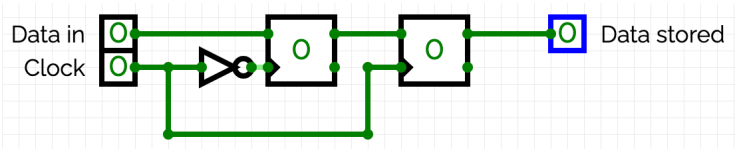


Remark
A useful rule of thumb is:
- If a circuit does not work in the simulator, it will not work in real hardware either.
- If a circuit works in the simulator, it may work in real hardware—but it is not guaranteed to. After all, every simulation is only a model of reality and therefore necessarily simplifies it.

Step 9

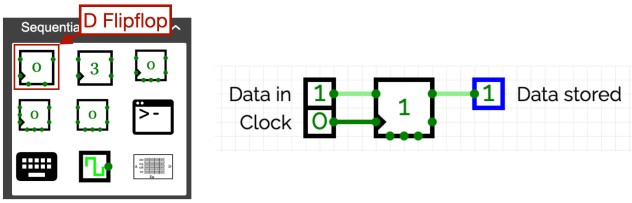
A D latch is said to be level-triggered, because its behavior depends on the current value of the Enable signal. Later in the course, however, we will use D flip-flops to store data. Unlike a D latch, a D flip-flop is edge-triggered.

To develop an intuition for the difference between level-triggered and edge-triggered storage elements, build the following circuit using two D latches and one NOT gate:



Experiment with the circuit and then explain to a partner (or to one of us) why Data in is stored on the rising edge of the Clock signal.

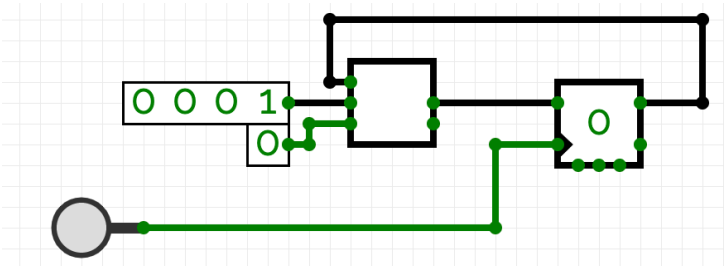
Finally, replace the circuit with the built-in D flip-flop and verify that it behaves in exactly the same way.



Step 10

Later in the course, we will use **D flip-flops** to build the CPU registers of our processor. Before doing so, however, let us look at a simple example illustrating how **combinational logic** (the adder) and **sequential logic** (the D flip-flop) work together.

Build the following circuit using a 4-bit adder together with a 4-bit D flip-flop.



When experimenting with the circuit, you should observe that each press of the button increments the value stored in the flip-flop.

This also provides some intuition for why the maximum clock frequency of a processor depends on the length of its longest logic path. If the propagation delay of the adder were modeled more accurately and the button (or clock) were triggered too quickly, the counter would no longer operate correctly.

Also convince yourself why a **D flip-flop** is required in this circuit and why replacing it with a **D latch** does not work. Replace the D flip-flop with a D latch and experiment with the circuit.

In the coming sessions, we will build on these ideas to construct a small ALU with registers, capable of executing a simple instruction set.